

Examples menu. Tower of Hanoi animations are readily found through an internet search as well.

The recursive algorithm for solving the Tower of Hanoi puzzle is not intuitive. Let's start with the smallest case: a Tower of Hanoi with one disk. The solution is trivial: move the disk to another pole and you're finished. Solving for two disks is slightly more complicated: move the smaller disk to one pole (we'll call it the *temporary pole*) and the larger disk to the other pole (we'll call it the *end pole*), and then finally move the smaller disk from the temporary pole to the end pole. Both disks are now on the end pole in the correct order.

Once you solve the three-disk tower, you'll notice that a pattern emerges. To solve a tower of n disks from the start pole to the end pole, you must do the following:

1. Solve the $n - 1$ disks puzzle by moving those disks from the start pole to the temporary pole.
2. Move the n th disk from the start pole to the end pole.
3. Solve the $n - 1$ disks puzzle by moving those disks from the temporary pole to the end pole.

Like the Fibonacci algorithm, the recursive case for the Tower of Hanoi algorithm makes two recursive calls instead of just one. If we draw a tree diagram of the operations for solving a four-disk Tower of Hanoi, it looks like [Figure 3-6](#). Solving the four-disk puzzle requires the same steps as solving the three-disk puzzle, as well as moving the fourth disk and performing the steps of solving the three-disk puzzle again. Likewise, solving the three-disk puzzle requires the same steps as the two-disk puzzle plus moving the third disk, and so on. Solving the one-disk puzzle is the trivial base case: it involves only moving the disk.

The tree-like structure in [Figure 3-5](#) hints that a recursive approach is ideal for solving the Tower of Hanoi puzzle. In this tree, the execution moves from top to bottom and from left to right.

While a three-disk or four-disk Tower of Hanoi is easy for a human to solve, increasing numbers of disks require an exponentially increasing number of operations to complete. For n disks, it takes a minimum of $2^n - 1$